Compte rendu - TP1 d'Ordonnancement temps réel

Bongiovanni Arthur
3A SN - (E)

Exercice 1

1.a

Cette configuration est ordonnançable en utilisant l'algorithme Deadline Monotonic (DM).
En effet, les tâches ont une priorité fixe :
T2 est plus prioritaire que T1 car (D_2 = 4 < D_1 = 5).

Nous pouvons vérifier l'ordonnançabilité à l'aide de l'analyse du temps de réponse (RTA) :

$$R_1^{(0)} = C_1 = 2$$

$$R_1^{(1)} = C_1 + \lceil \frac{R_1^{(0)}}{P_2} \rceil C_2 = 2 + \lceil \frac{2}{10} \rceil \times 2 = 4$$

$$R_1^{(2)} = 2 + \lceil \frac{4}{10} \rceil \times 2 = 4 \Rightarrow R_1 = 4 \leq D_1 = 5$$

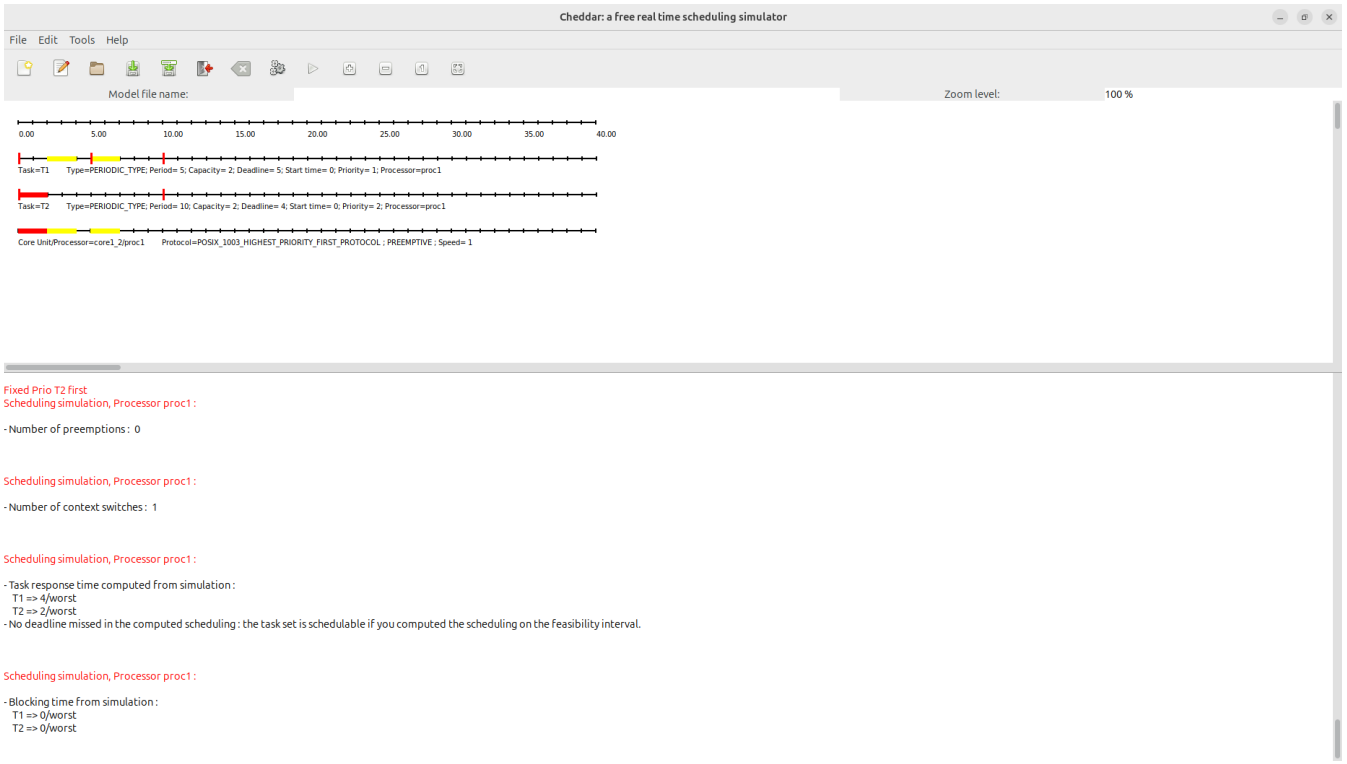
Pour T2 (plus haute priorité, pas d'interférence) :

$$R_2 = C_2 = 2 \leq D_2 = 4$$

Le système est donc ordonnançable sous Deadline Monotonic.

Avec la simulation, on confirme que le système respecte les échéances :

- T1 atteint un pire temps de réponse de 4
- T2 un pire temps de réponse de 2.



1.b

En inversant les priorités, on obtient l'ordonnancement Rate Monotonic :
T1 est plus prioritaire que T2 car (P_1 = 5 < P_2 = 10).

Le test d'ordonnançabilité donne :

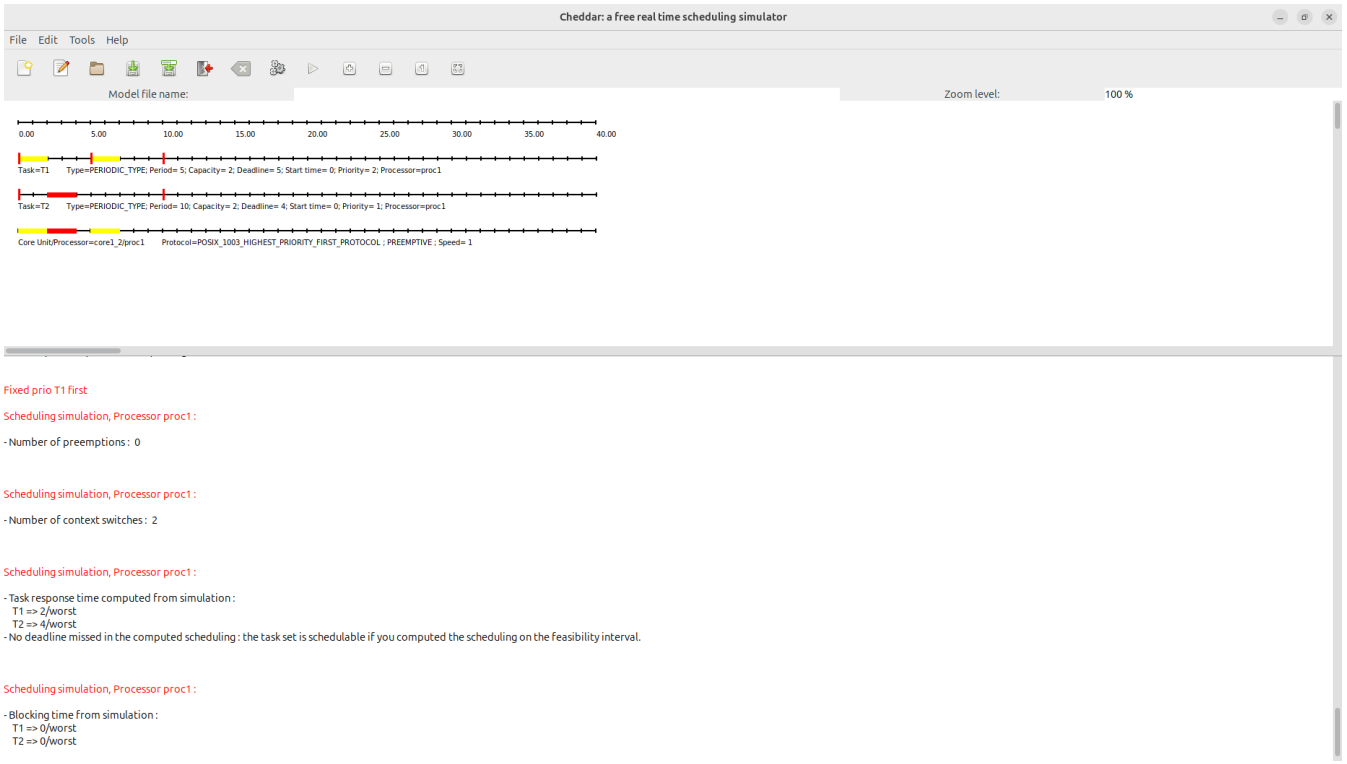
$$R_1 = C_1 = 2 \leq D_1 = 5$$

$$R_2^{(0)} = C_2 = 2$$

$$R_2^{(1)} = C_2 + \lceil \frac{R_2^{(0)}}{P_1} \rceil C_1 = 2 + \lceil \frac{2}{5} \rceil \times 2 = 4$$

$$R_2^{(2)} = 2 + \lceil \frac{4}{5} \rceil \times 2 = 4 \Rightarrow R_2 = 4 \leq D_2 = 4$$

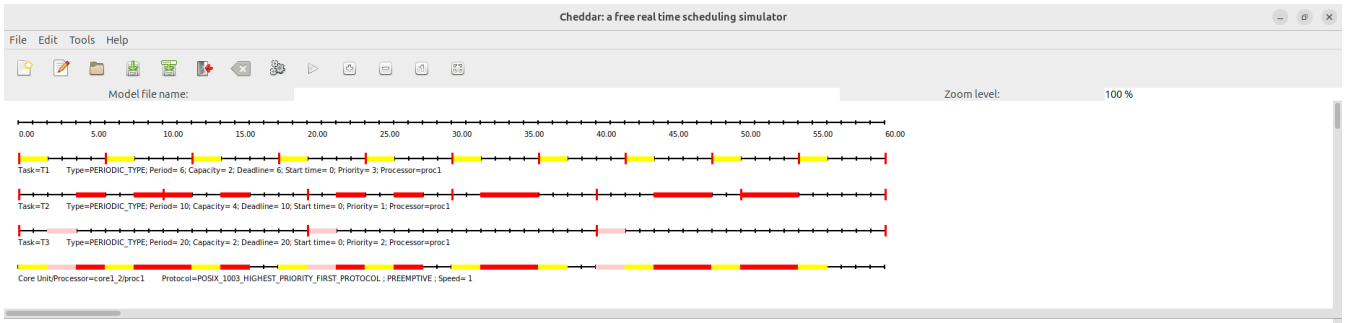
Le système est donc aussi ordonnançable sous Rate Monotonic.



Exercice 2

1

En utilisant Rate Monotonic, la configuration est ordonnançable avec les priorités fixes : (T1 > T2 > T3).
(nb: la capture d'écran pour ces priorité est corrompu mais celle pour T1 > T3 > T2 est intact, donc comme cette simulation montre aussi un sytème à priorité fixe qui fonctionne pour cette configuration, je l'ai mis à la place)



Fixed prio t1 > t3 > t2

Scheduling simulation, Processor proc1 :

- Number of preemptions : 3

Scheduling simulation, Processor proc1 :

- Number of context switches : 20

Scheduling simulation, Processor proc1 :

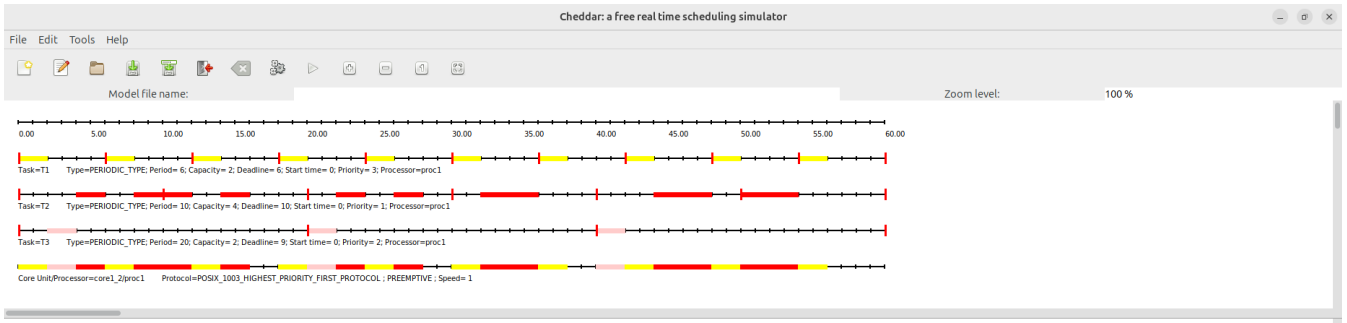
- Task response time computed from simulation :
  T1 => 2/worst
  T2 => 10/worst
  T3 => 4/worst
- No deadline missed in the computed scheduling : the task set is schedulable if you computed the scheduling on the feasibility interval.

Scheduling simulation, Processor proc1 :

- Blocking time from simulation :
  T1 => 0/worst
  T2 => 0/worst
  T3 => 0/worst

2

En utilisant **Deadline Monotonic**, la configuration est également ordonnançable, avec priorités :
(T1 > T3 > T2).



Fixed prio : t1 > t3 > t2

Scheduling simulation, Processor proc1 :

- Number of preemptions : 3

Scheduling simulation, Processor proc1 :

- Number of context switches : 20

Scheduling simulation, Processor proc1 :

- Task response time computed from simulation :
  T1 => 2/worst
  T2 => 10/worst
  T3 => 4/worst
- No deadline missed in the computed scheduling : the task set is schedulable if you computed the scheduling on the feasibility interval.

Scheduling simulation, Processor proc1 :

- Blocking time from simulation :
  T1 => 0/worst
  T2 => 0/worst
  T3 => 0/worst

3

En utilisant un algorithme à priorité fixe, la tâche de plus faible priorité rate sa deadline.

Cas A — T1 plus basse priorité

- Interférences dans [0, D1) = [0, 6) :
  - T2 : ⌈6/10⌉ = 1 job → 1 × C2 = 4
  - T3 : ⌈6/20⌉ = 1 job → 1 × C3 = 2
  - Total interférence = 6
    - Temps dispo = 6 − 6 = 0 < C1 = 2
    - T1 rate sa deadline

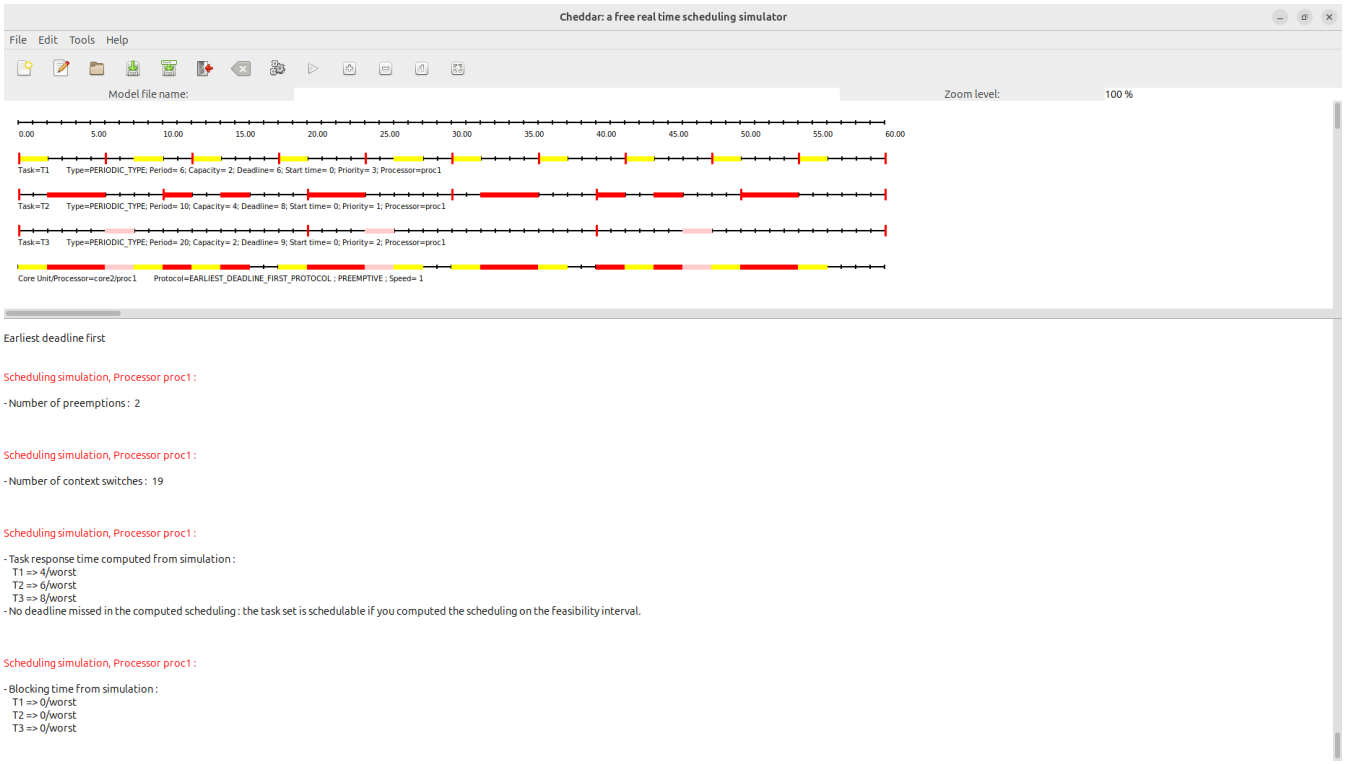
Cas B — T2 plus basse priorité

- Interférences dans [0, D2) = [0, 8) :
  - T1 : ⌈8/6⌉ = 2 → 2 × C1 = 4
  - T3 : ⌈8/20⌉ = 1 → 1 × C3 = 2
  - Total interférence = 6
    - Temps dispo = 8 − 6 = 2 < C2 = 4
    - T2 rate sa deadline

Cas C — T3 plus basse priorité

- Interférences dans [0, D3) = [0, 9) :
  - T1 : ⌈9/6⌉ = 2 → 2 × C1 = 4
  - T2 : ⌈9/10⌉ = 1 → 1 × C2 = 4
  - Total interférence = 8
    - Temps dispo = 9 − 8 = 1 < C3 = 2
    - T3 rate sa deadline

Cependant, avec un algorithme à priorité dynamique (Earliest Deadline First), le système devient ordonnançable.

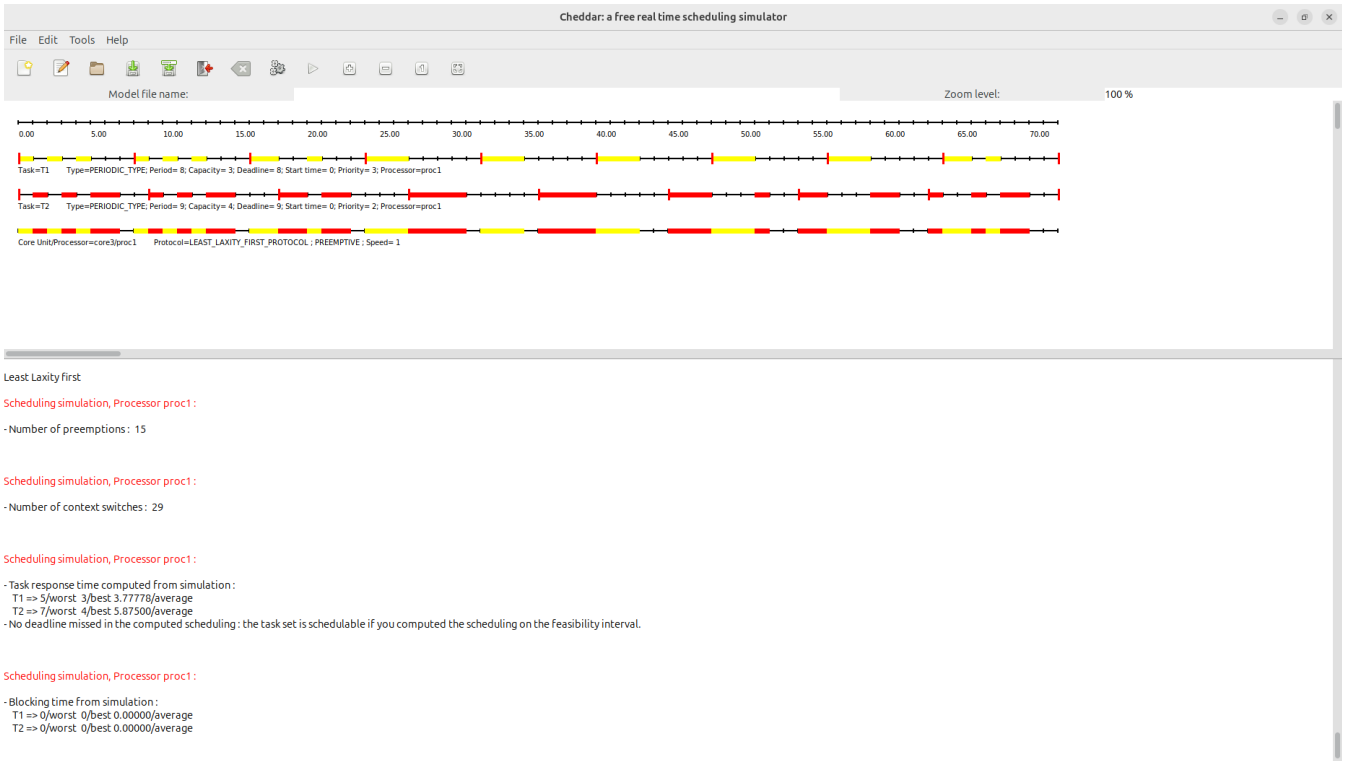


Exercice 3

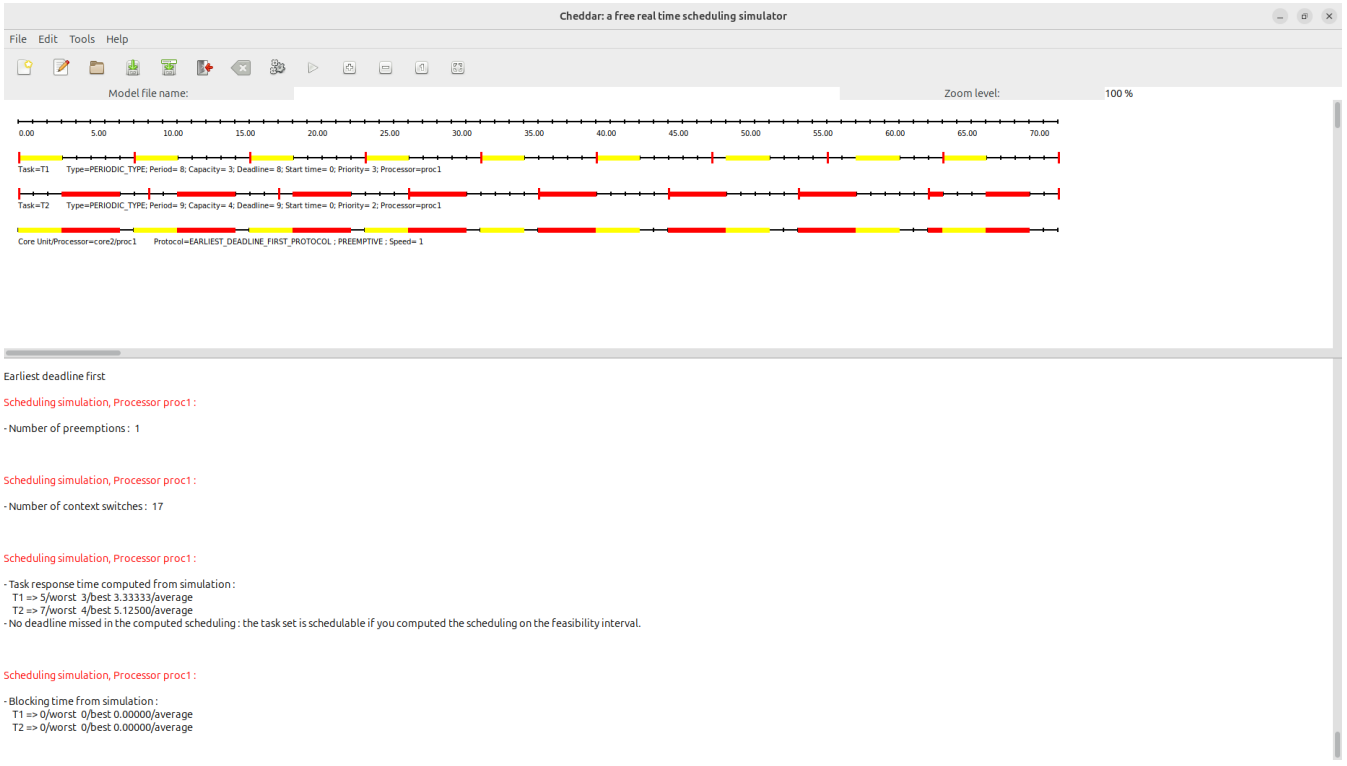
Le système est ordonnançable avec les deux algorithmes (EDF et LLF).
Cependant, LLF engendre un nombre de préemptions bien plus important et donc un temps moyen d'exécution plus élevé.

Il est donc plus intéressant d'utiliser Earliest Deadline First (EDF) pour ce système.
À noter : la simulation ne prend pas en compte le coût de préemption, ce qui pourrait, dans la réalité, conduire à des dépassements d'échéance sous LLF.

simulation LLF



simulation EDF



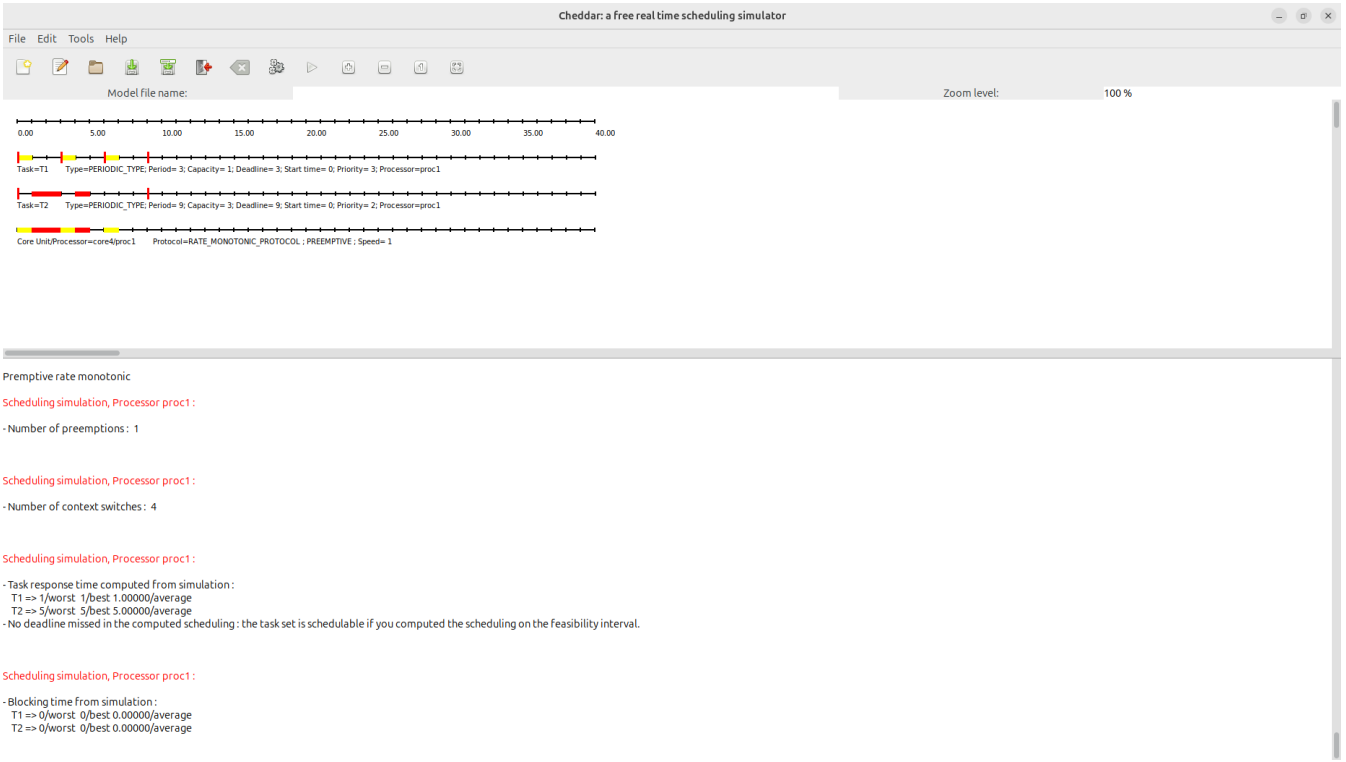
Exercice 4

1

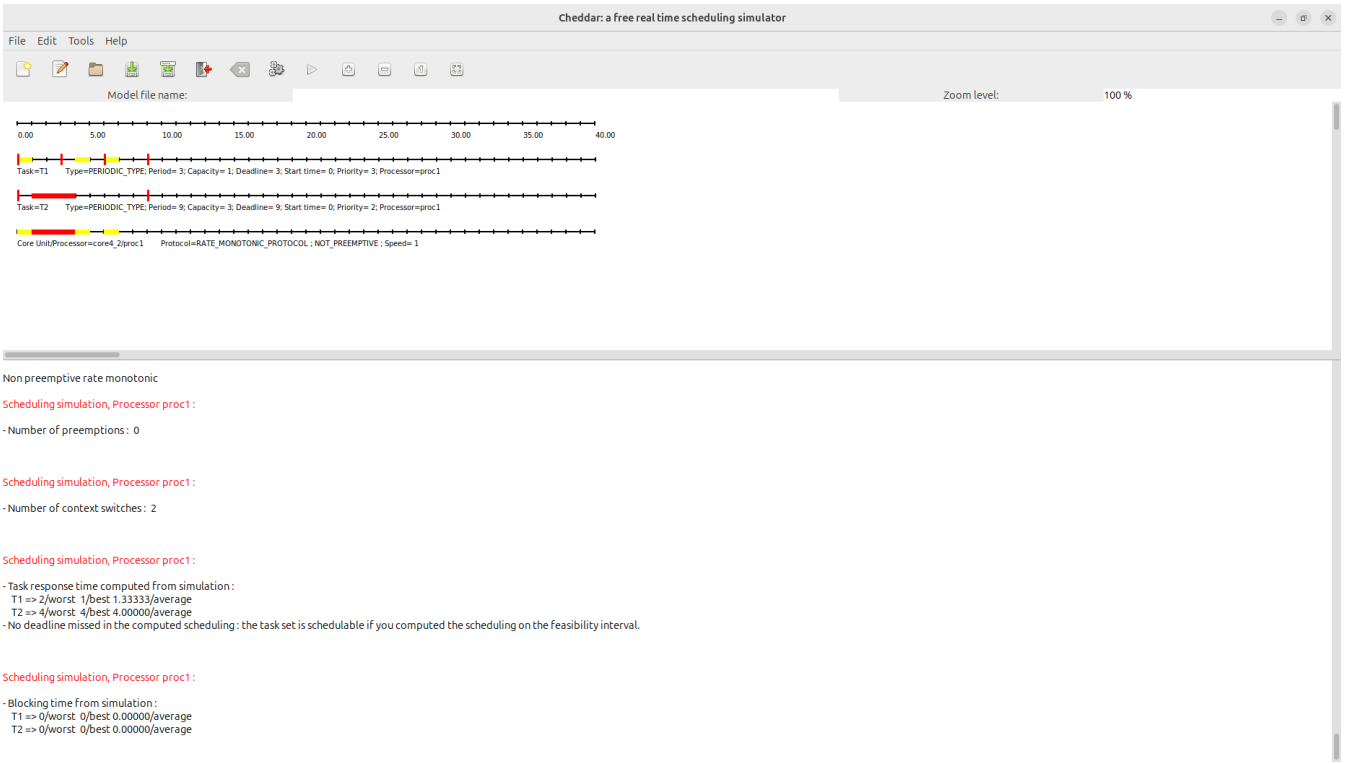
Avec et sans préemption, aucune tâche ne dépasse son échéance.
Cependant, avec préemption, les tâches sont exécutées avec des temps d'exécution constants :

(worst time = best time).

avec préemption



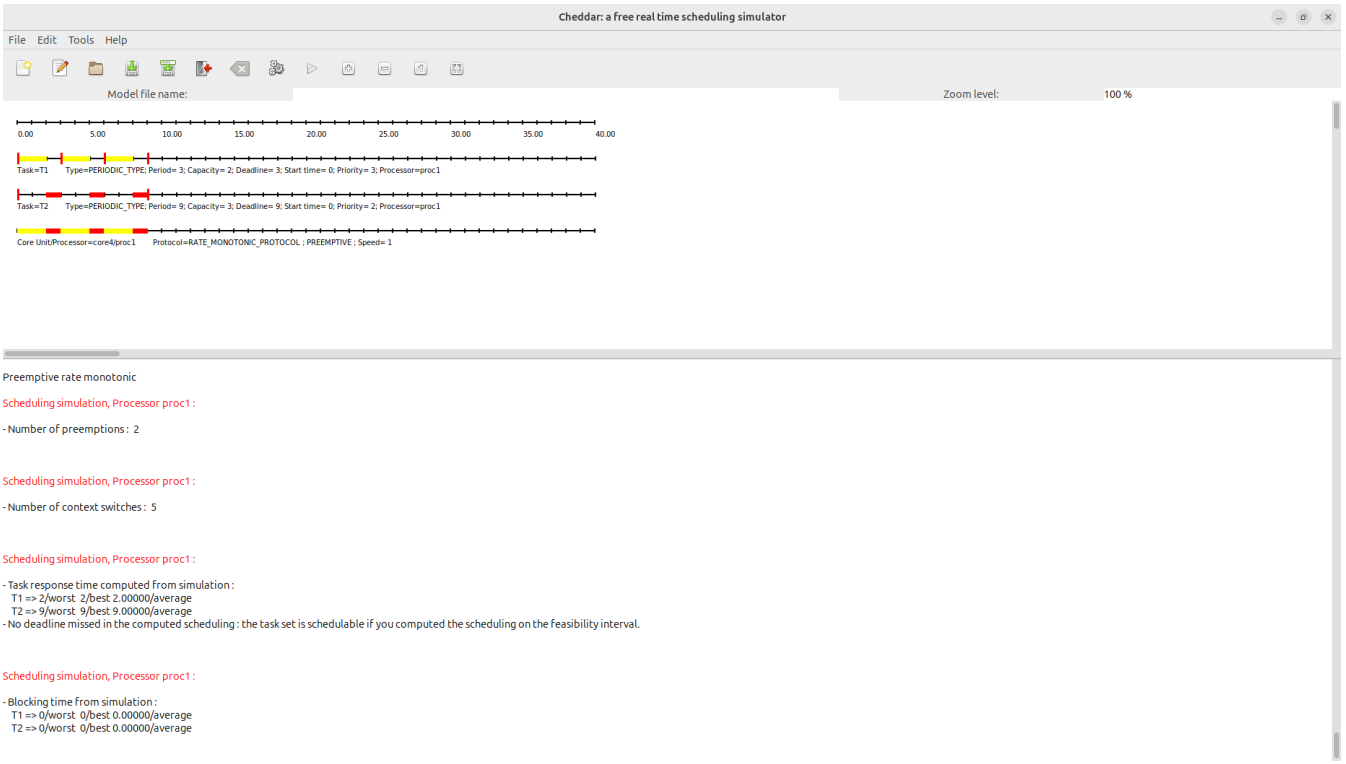
sans préemption



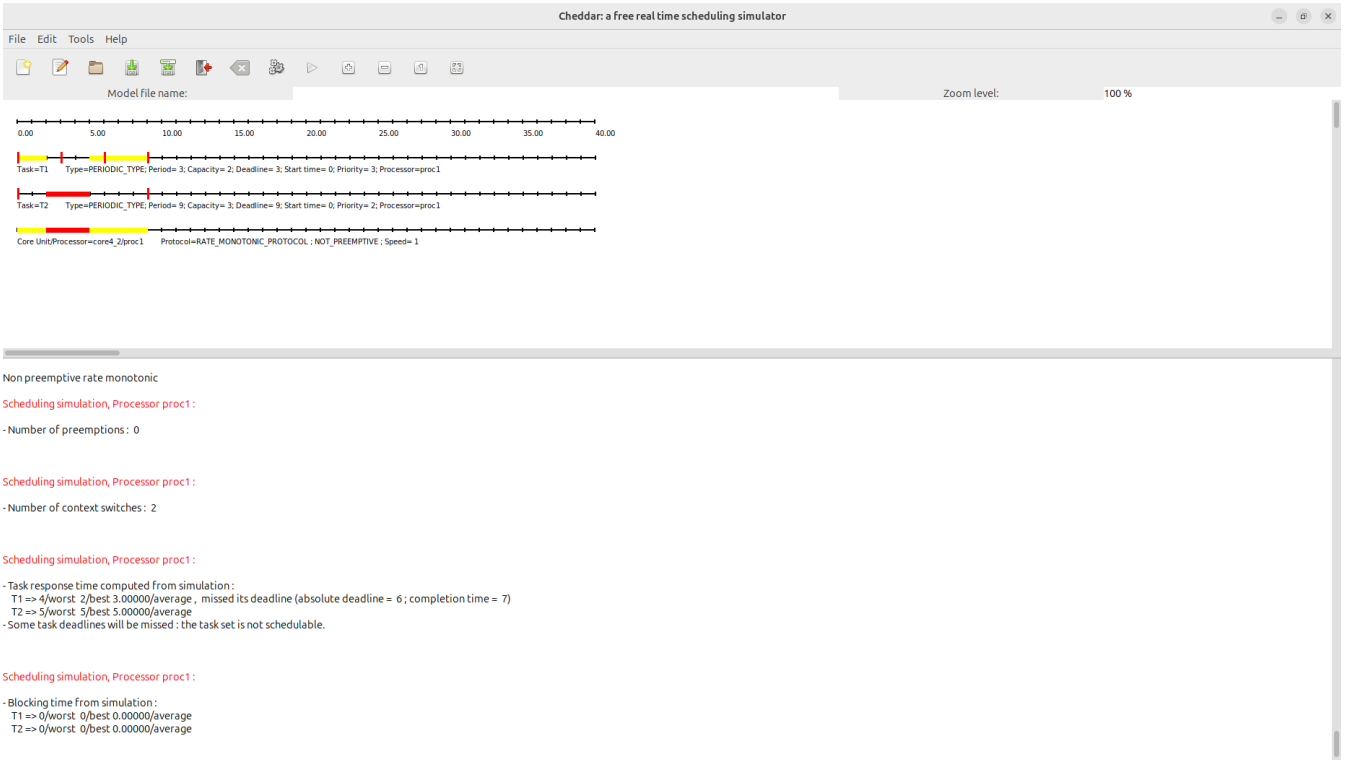
2

En augmentant le temps d'exécution de T1, le système n'est plus ordonnançable sans préemption, alors qu'il reste ordonnançable en mode préemptif.

avec préemption



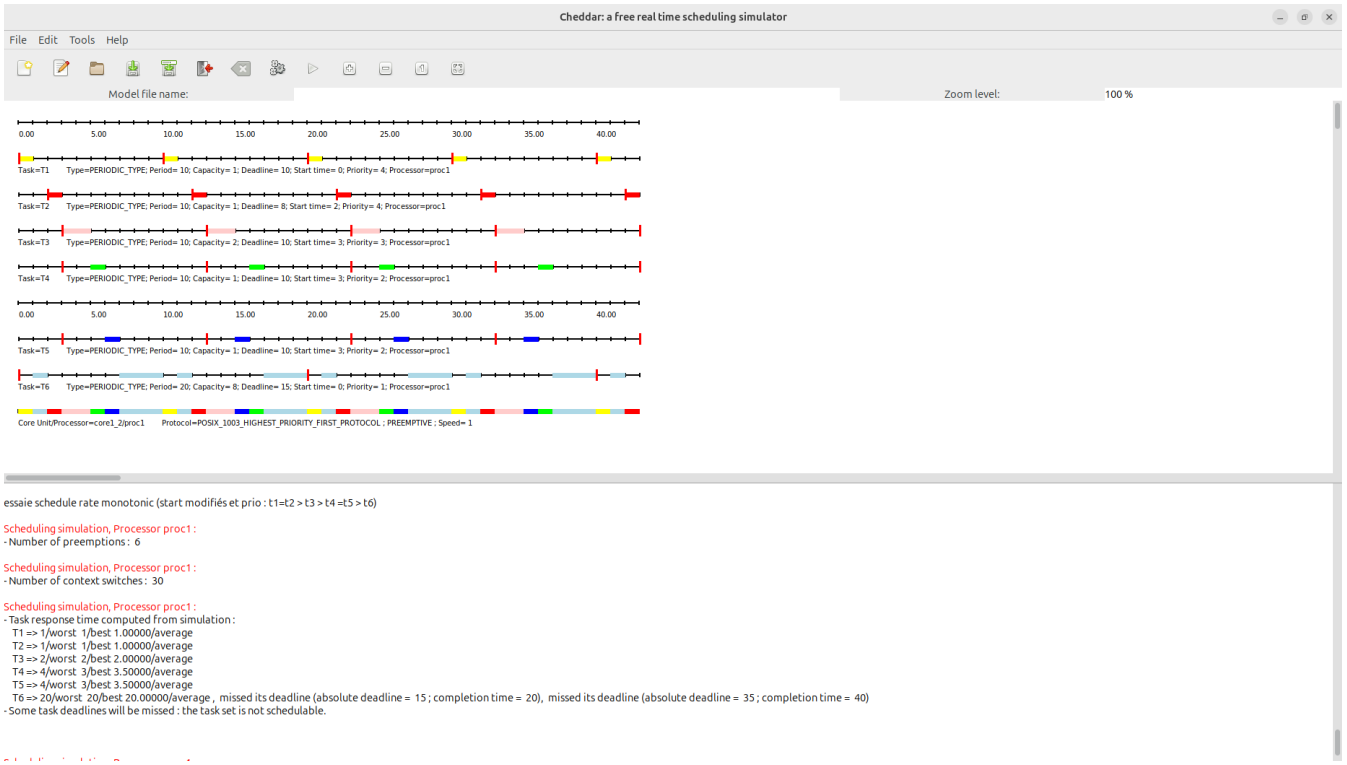
sans préemption



Exercice 5

1

En modifiant uniquement les dates de premier appel et en prenant les priorités suivantes : (T1 = T2 > T3 > T4 = T5 > T6),
le système n'est pas ordonnançable : la tâche T6 rate son échéance.



2

En conservant les mêmes dates de premier appel mais en utilisant EDF, le système devient ordonnançable.

